BICOL UNIVERSITY POLANGUI / COMPUTER STUDIES DEPARTMENT

Phase 3
Frontend Development

BU Student Academic Task and Deadline Management System

Group 5

BSIS-2B / IT 112 Web Systems and Technologies

Bea Roshan O. Pabilona

Eleanor Aimee O. Reantaso

Christian A. Balauro

Simranjit S. Singh

Karl James Matias

Submitted to

Guillermo V. Red, Jr.

Instructor

Phase 1 design alignment

Source: Group 5 Phase 1 proposal, use case diagram, ERD, and architecture (pp. 3-7).

Student (Users.role = user)

Register or log in, view deadlines, manage personal tasks, assign an optional subject, update status, and search or filter tasks.

Administrator (Users.role = admin)

Manage user accounts and view all tasks with their owners for system oversight.

Users collection

name and **email** are entered on registration. The password input maps to **passwordHash** through future server-side hashing. The server assigns the default user role and manages identifiers and timestamps.

Tasks and Categories collections

title and **dueDate** are required. **description** is optional. **status** and **priority** use the ERD values. The optional **category** references **Categories._id**. The subject form collects **name** and optional **color**.

Navigation

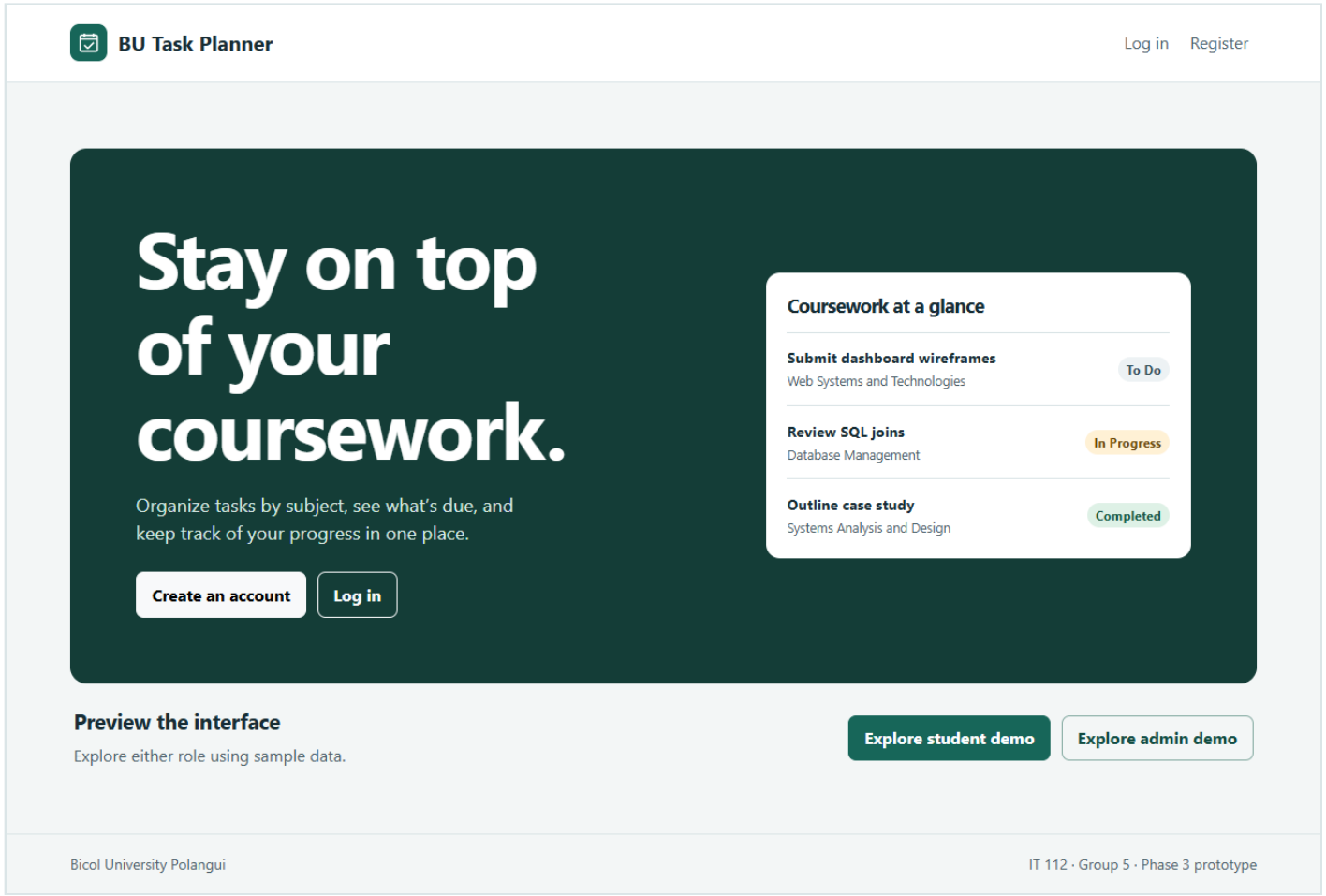
Home links to registration, login, and both sample role previews. The student dashboard links to the task list and Profile. The administrator page groups account management and task oversight.

Architecture handoff

This lab implements the browser and HTML/CSS/Bootstrap layer. A shared JavaScript submit handler marks future Fetch requests to Express. Express will validate identity, roles, ownership, and inputs before accessing MongoDB Atlas through Mongoose.

Home

frontend/index.html

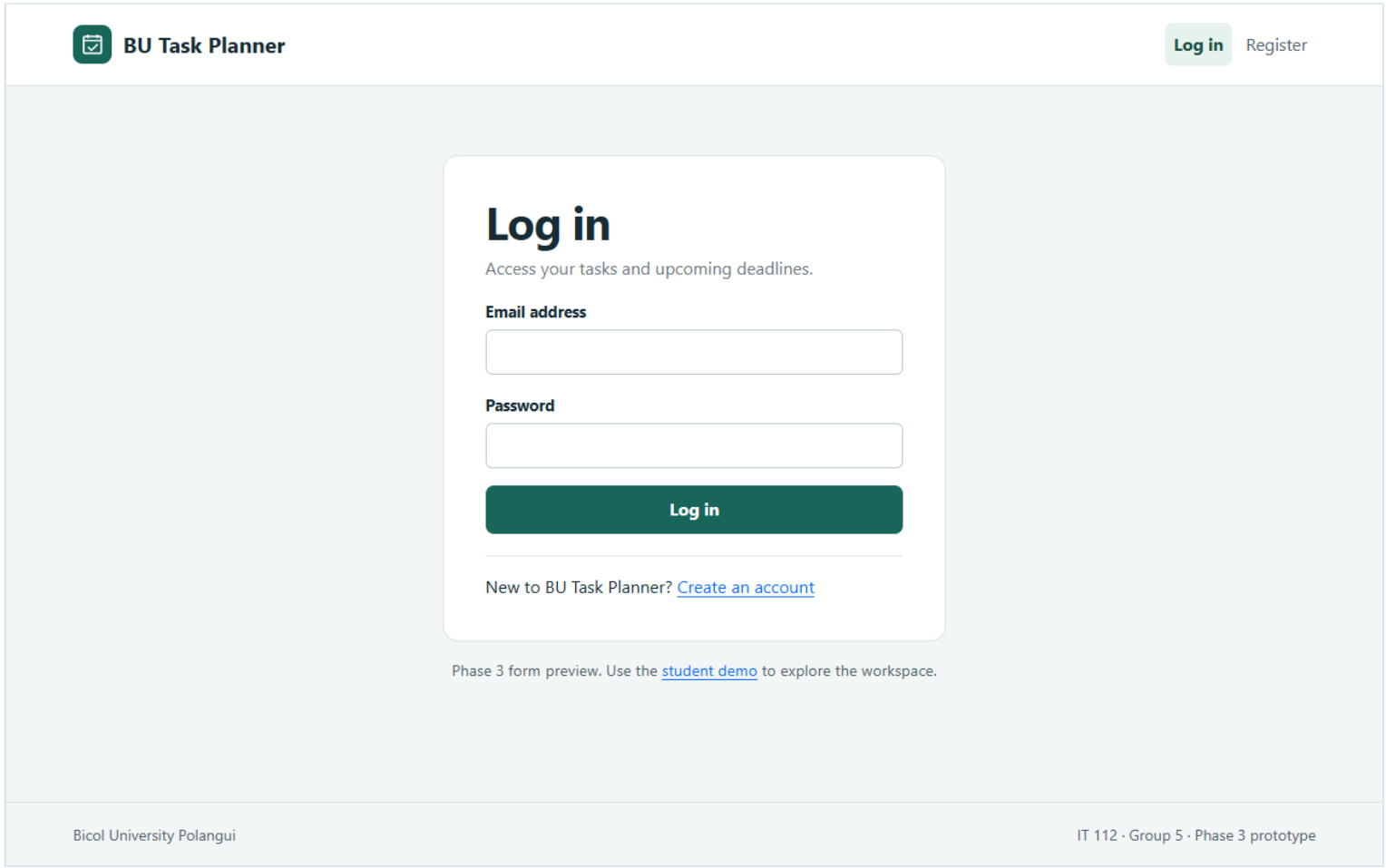


Public entry point

Registration and login links lead to the account forms. The demo links open the student dashboard or administration page.

Log in

frontend/login.html

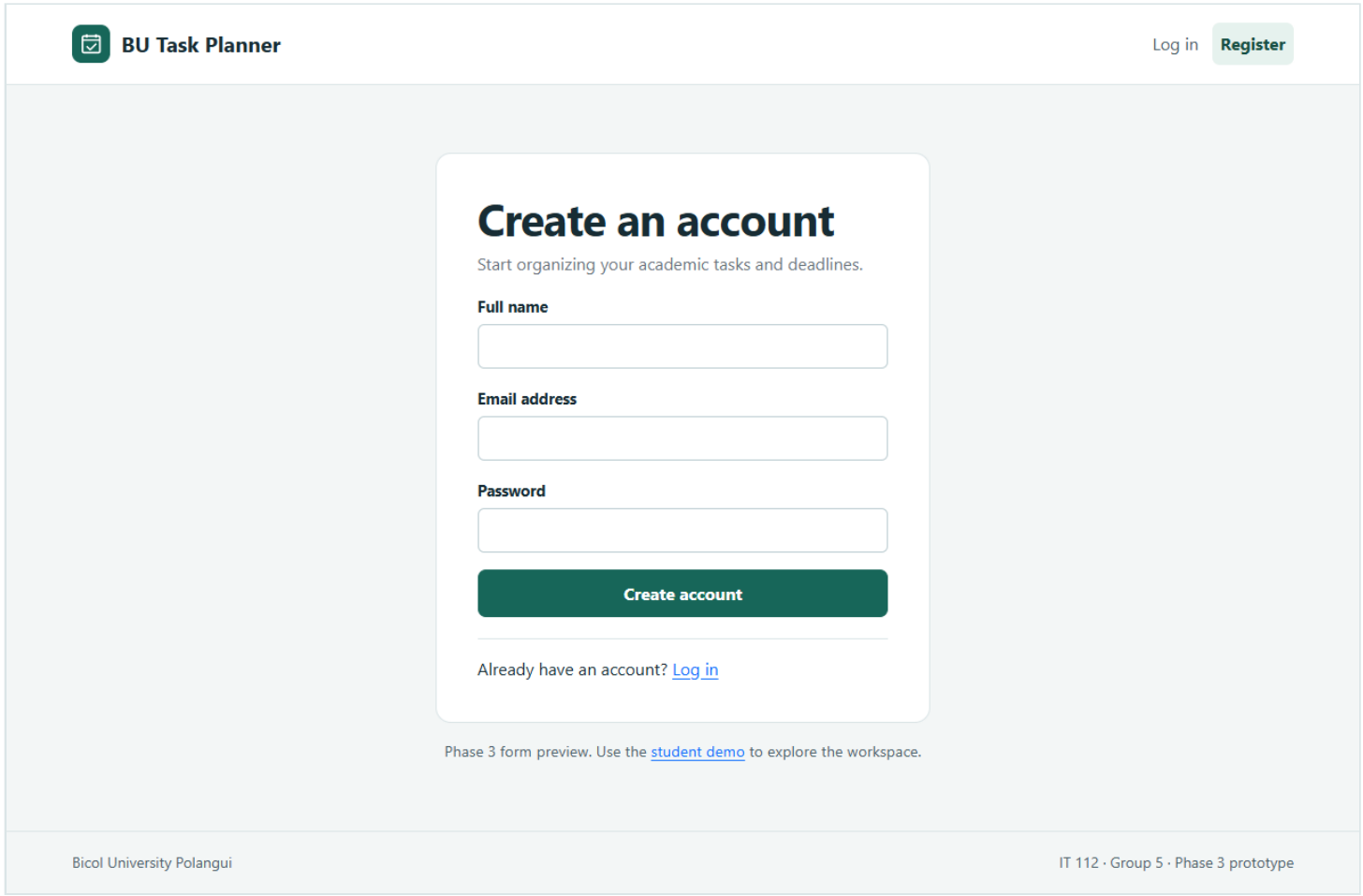


Register / log in

The form collects email and password. A future login endpoint will verify the password against Users.passwordHash and identify the role.

Register

frontend/register.html

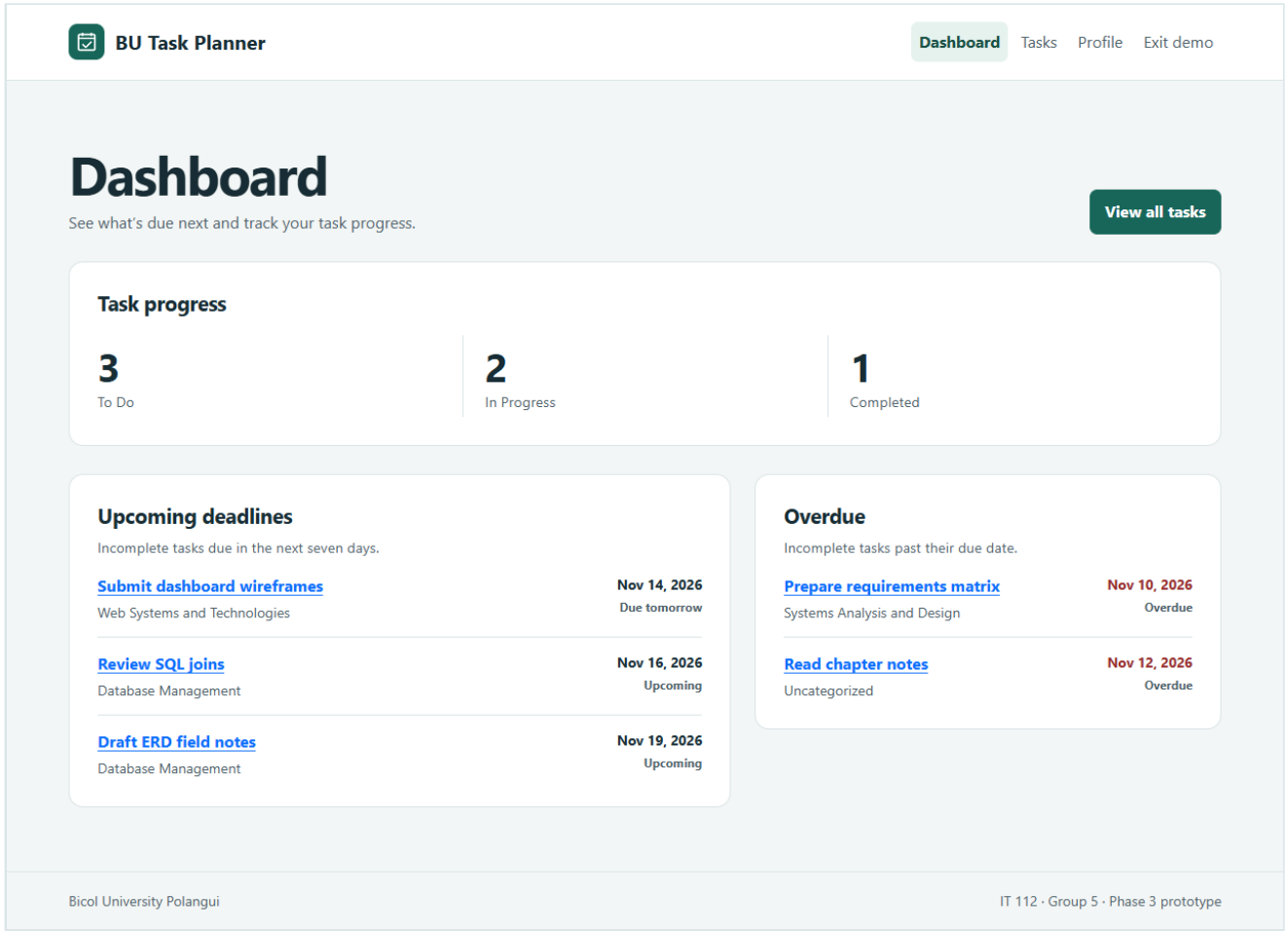


Register / log in

A student enters name, email, and password. The future API will assign the user role and manage hashing and account metadata.

Student dashboard

frontend/dashboard.html

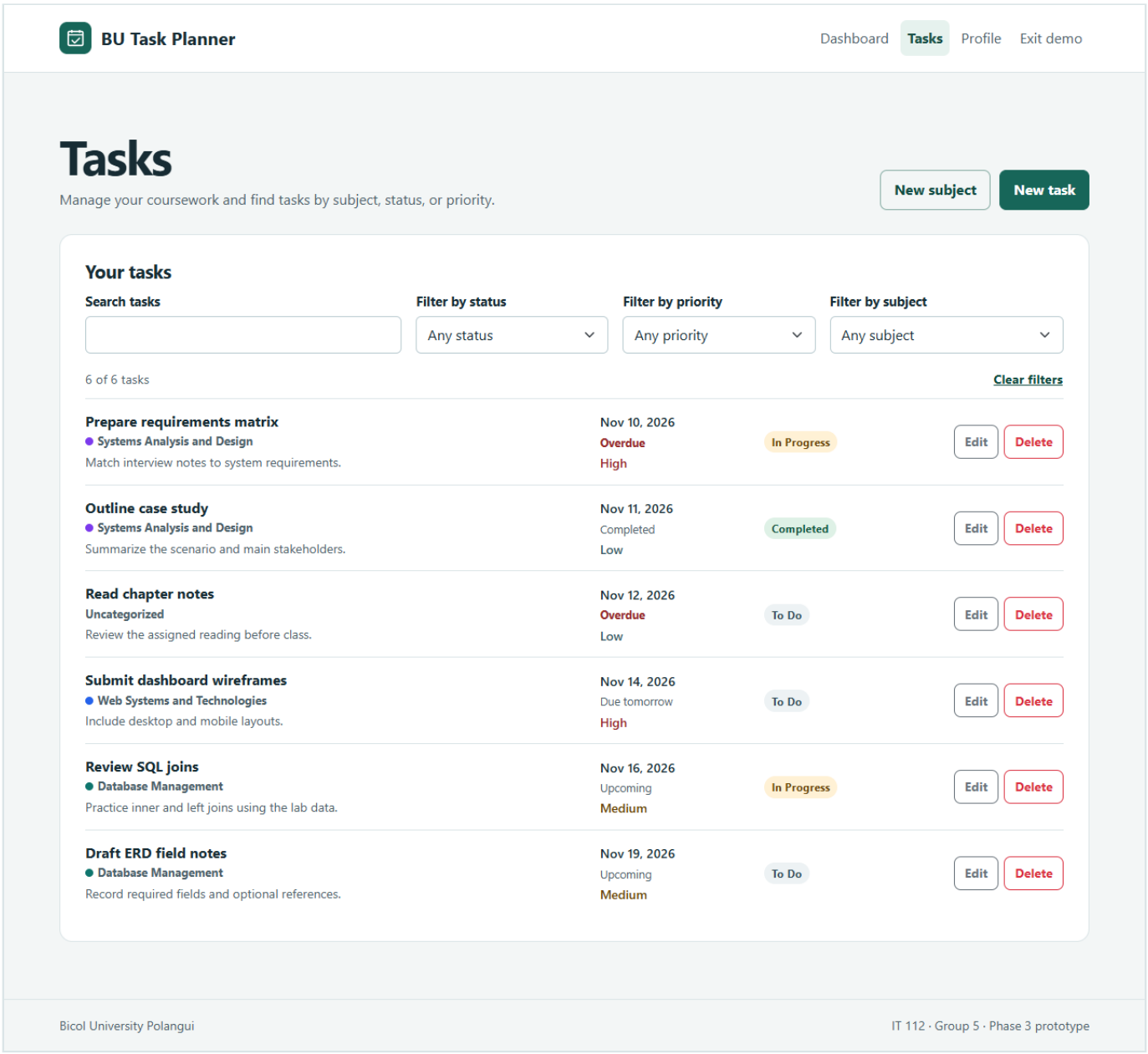


View dashboard and deadlines

Status totals use Tasks.status. The deadline cards use Tasks.dueDate to show incomplete tasks due within seven days or already overdue.

Tasks

frontend/tasks.html

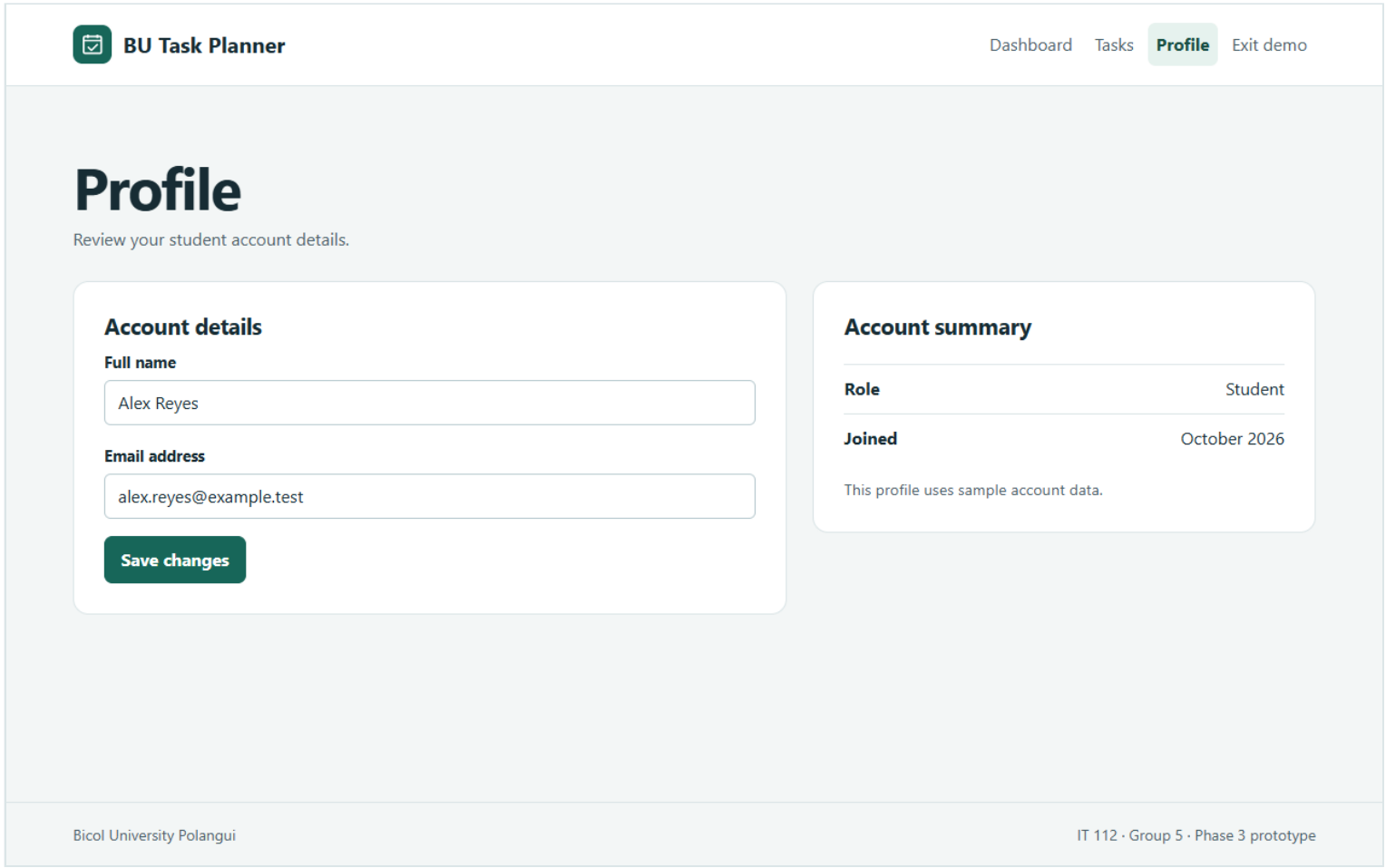


Manage, search, and filter personal tasks

Task rows show title, description, subject, due date, priority, and status. Keyword, status, priority, and subject filters work on the sample records.

Profile

frontend/profile.html

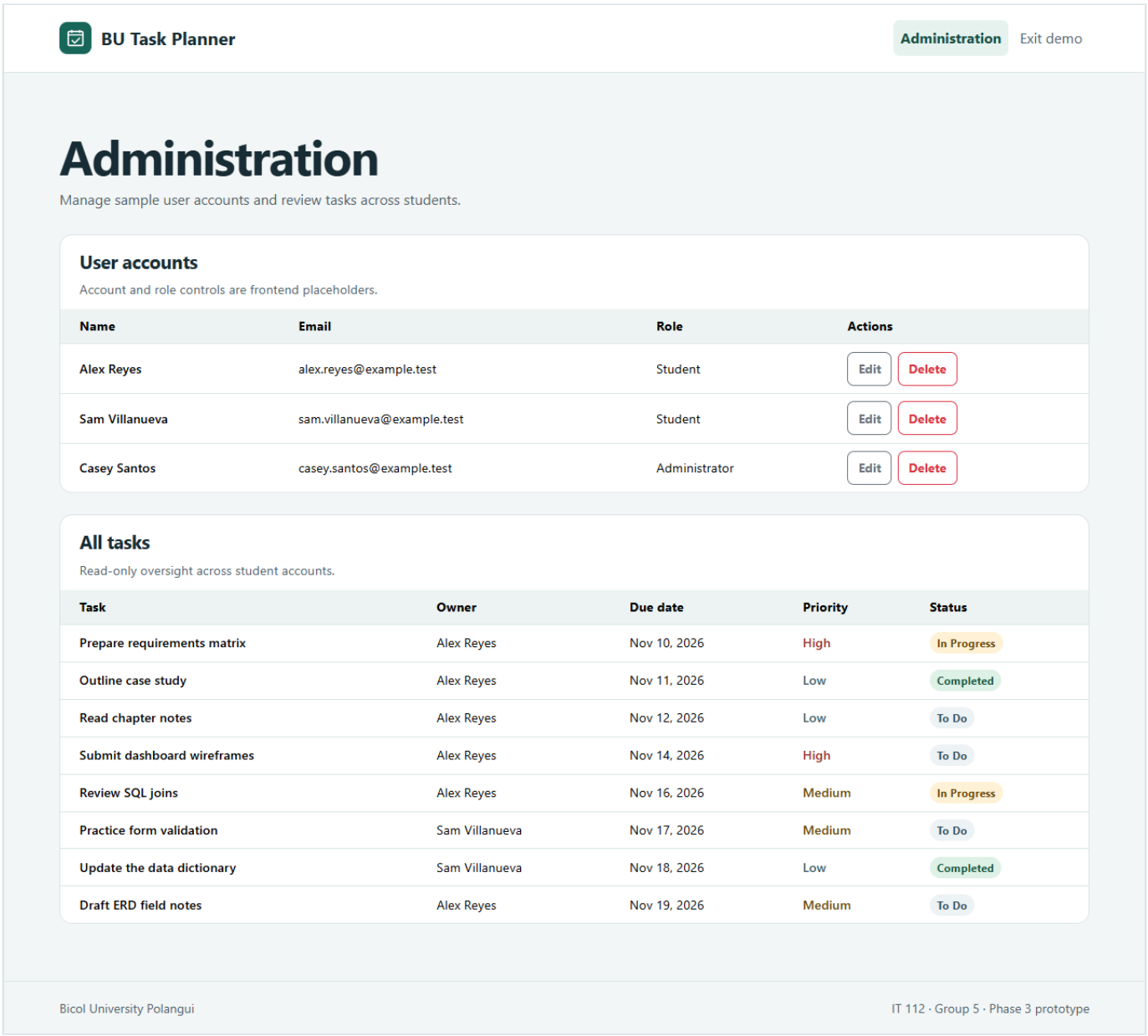


Review the student account

Name and email map to Users.name and Users.email. The account summary displays role and join date as read-only information.

Administration

frontend/admin.html



Manage accounts and oversee all tasks

The account table shows name, email, and role. Edit and delete dialogs carry the record ID. The oversight table shows sample tasks from multiple student owners.

Form fields and data ownership

Field-by-field mapping to the Phase 1 MongoDB ERD

Users

_id identifies the selected account in admin dialogs. **name** and **email** appear in account forms. **passwordHash** will be produced by the backend from the password input. **role** uses user or admin. **createdAt** and **updatedAt** are server-managed timestamps.

Tasks

_id identifies the selected task for edit or delete. **title**, **description**, **dueDate**, **priority**, and **status** appear in the task form. **category** stores an optional category reference. **owner** will come from the authenticated user. **createdAt** and **updatedAt** are server-managed.

Categories

name and optional **color** appear in the subject form. **_id** and **createdAt** are server-managed. **owner** will come from the authenticated user. A task can remain uncategorized.

Allowed values

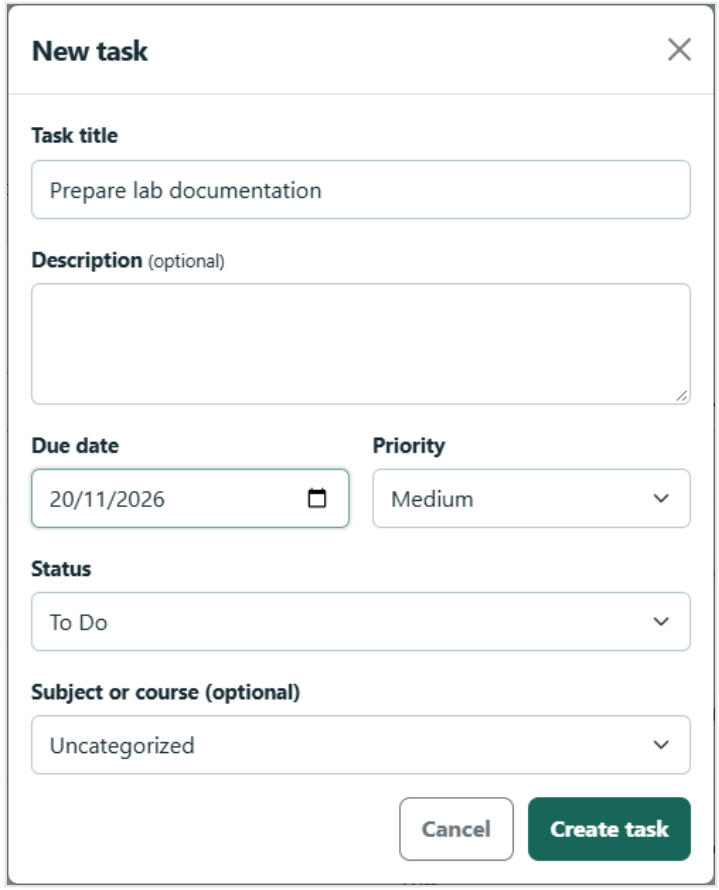
status uses To Do, In Progress, or Completed. **priority** uses Low, Medium, or High. Student registration defaults to the user role during future API integration.

Form conventions

Every form has an ID, action="#", and GET or POST method. Each input has a name and ID. Required fields use native browser validation. Whitespace-only task titles and names are rejected. Placeholder submissions display an action-specific message and move focus to it.

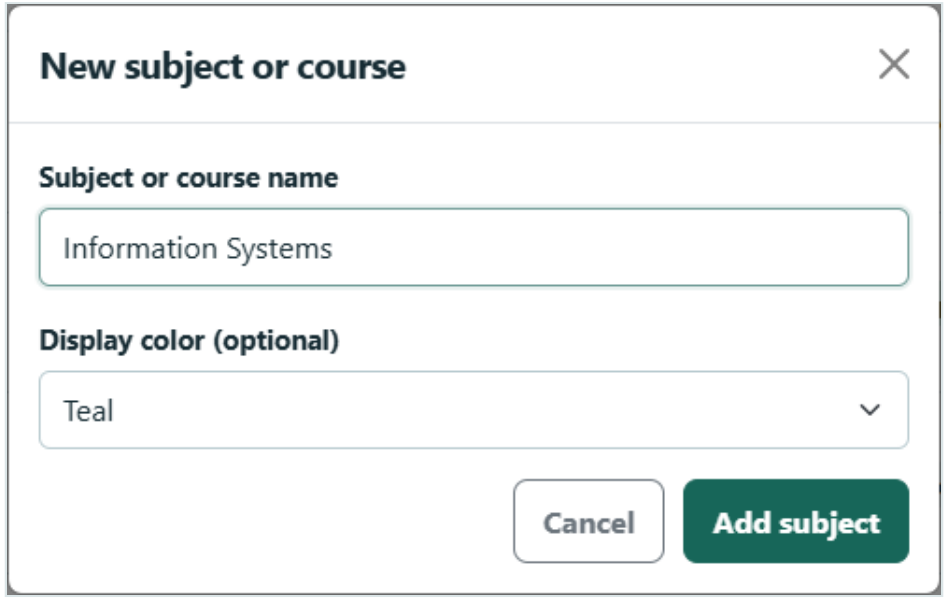
Create tasks and organize subjects

Bootstrap dialog evidence



Create a task

Required title and deadline, optional description and subject, and the exact ERD status and priority options.

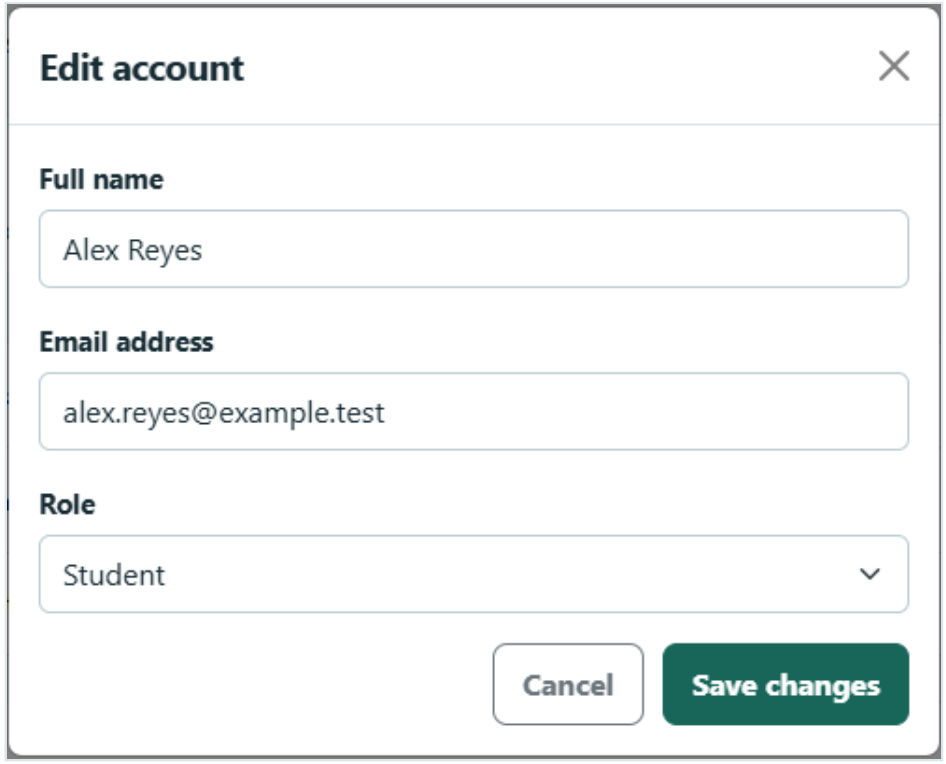


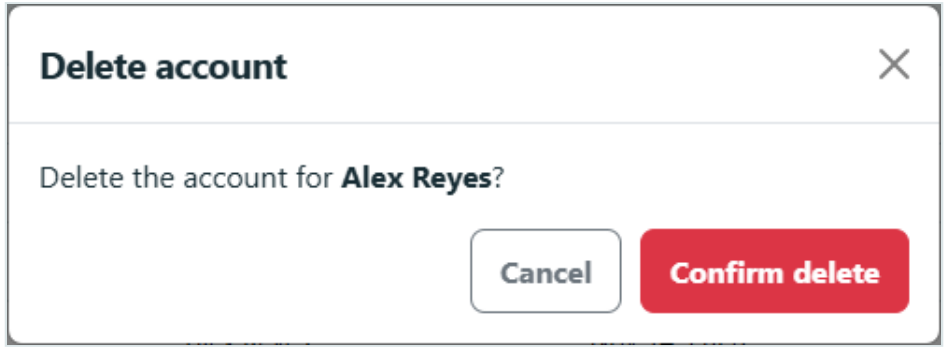
Create a subject

Subject name maps to Categories.name. The optional color value maps to Categories.color.

Account management and deletion

Bootstrap dialog evidence





Edit an account

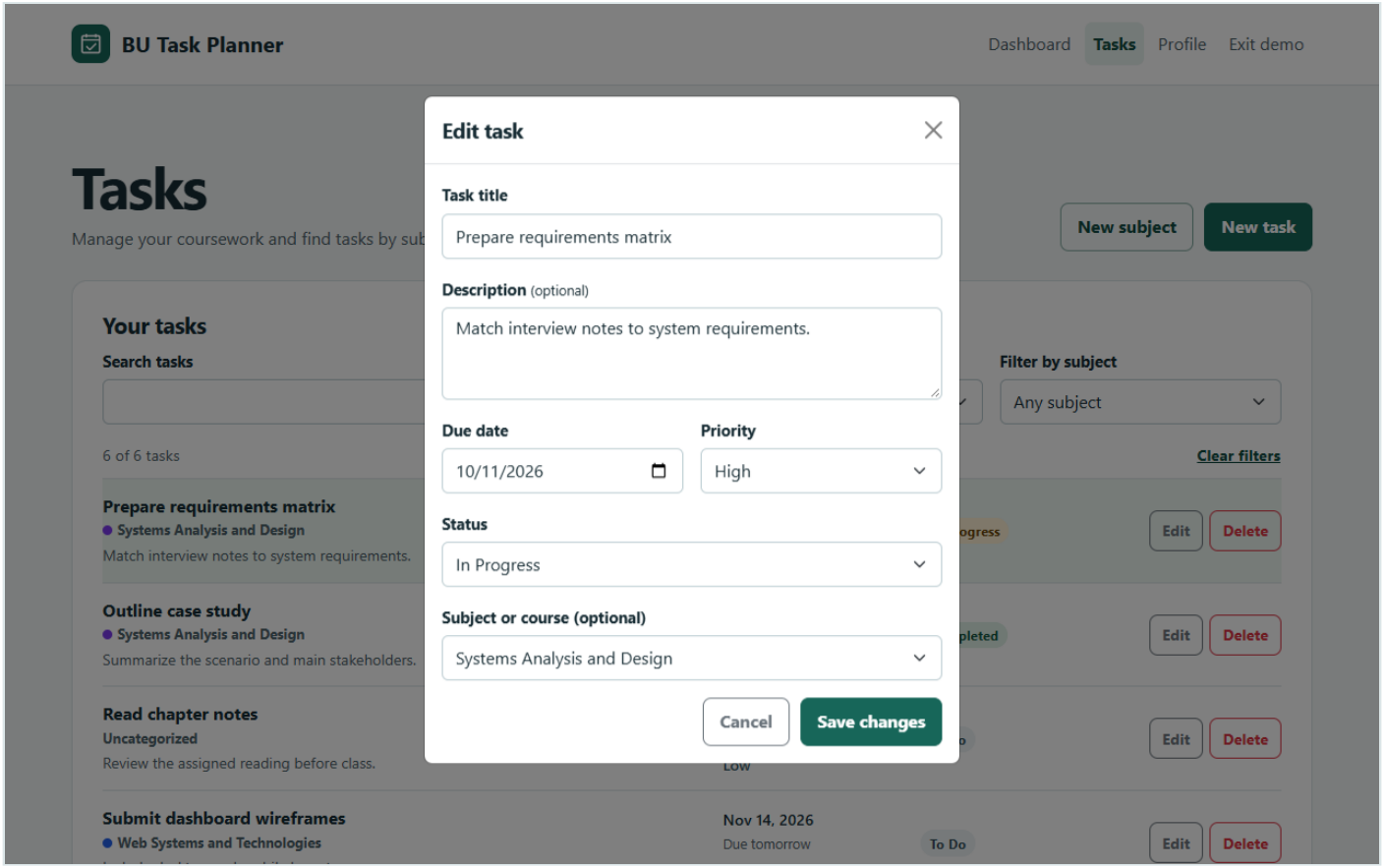
The administrator can review name, email, and role. The selected account ID is carried in the hidden _id input.

Delete an account

The confirmation identifies the selected account. Submitting displays the API placeholder message.

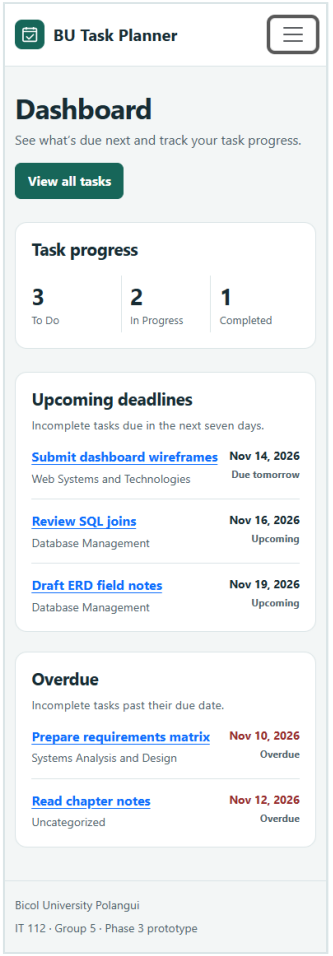
Edit task and mobile layout

Additional evidence for task field mapping and responsive navigation.



Edit task placeholder

The dialog preloads the selected task. It includes title, description, due date, priority, status, and optional subject. Submitting displays a preview message.

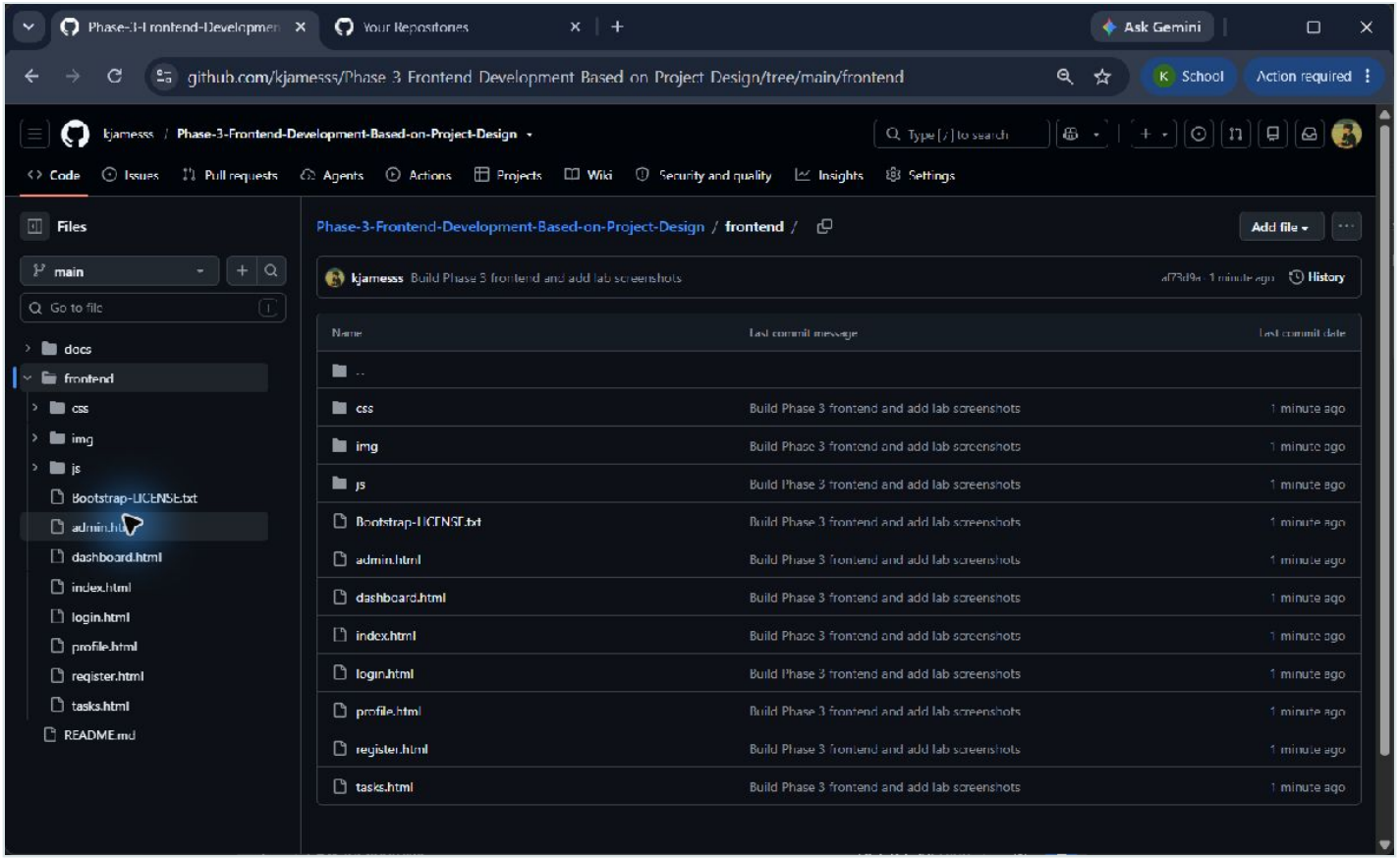


Mobile dashboard

The deadline cards stack and the navbar collapses.

Repository and commits

GitHub project evidence



Folder structure

GitHub file tree showing the submitted frontend and lab evidence.

af73d9a1
Build Phase 3 frontend and add lab screenshots

42d35aab
Clarify deadlines and validate form previews

fab6d2a4
Improve task scanning and keyboard focus

Open the GitHub repository

Individual reflections

Group 5

Bea Roshan O. Pabilona

Project Manager and Documentation Officer

I learned to turn the Phase 1 diagrams into a checklist for the frontend. The checklist made it easier to connect each screenshot to a student or administrator use case. In the next phase, I would use the same checklist to track which actions are ready for API testing.

Christian A. Balauro

Backend Developer / Back End-Manager

The form previews helped me see what the Express routes will need to accept. The edit and delete dialogs made it clear that the server will need a record ID and an ownership or role check. For integration, I would start with authentication and then test what happens when a user tries to access a task they do not own.

Karl James Matias

GitHub Manager

A consistent folder structure makes the frontend, screenshots, and report easier to find in GitHub. I learned why focused commit messages make it easier to trace the code and documentation changes. For the next phase, I would review the branch history before merging integration work.

Eleanor Aimee O. Reantaso

Front-End Developer / Front-End Manager

This phase showed me how a shared navbar and consistent form layout make separate pages feel like one application. I saw how much room the task filters and action buttons need when the layout becomes narrow. I would carry the same layout into the next phase and add feedback for API errors.

Simranjit S. Singh

Database Manager

Mapping the screens to the ERD helped me see which values users supply and which values the database layer manages. The optional subject field is a good example because a task must still be valid without a category reference. In the next phase, I would verify that the allowed status and priority values match the Mongoose schema.